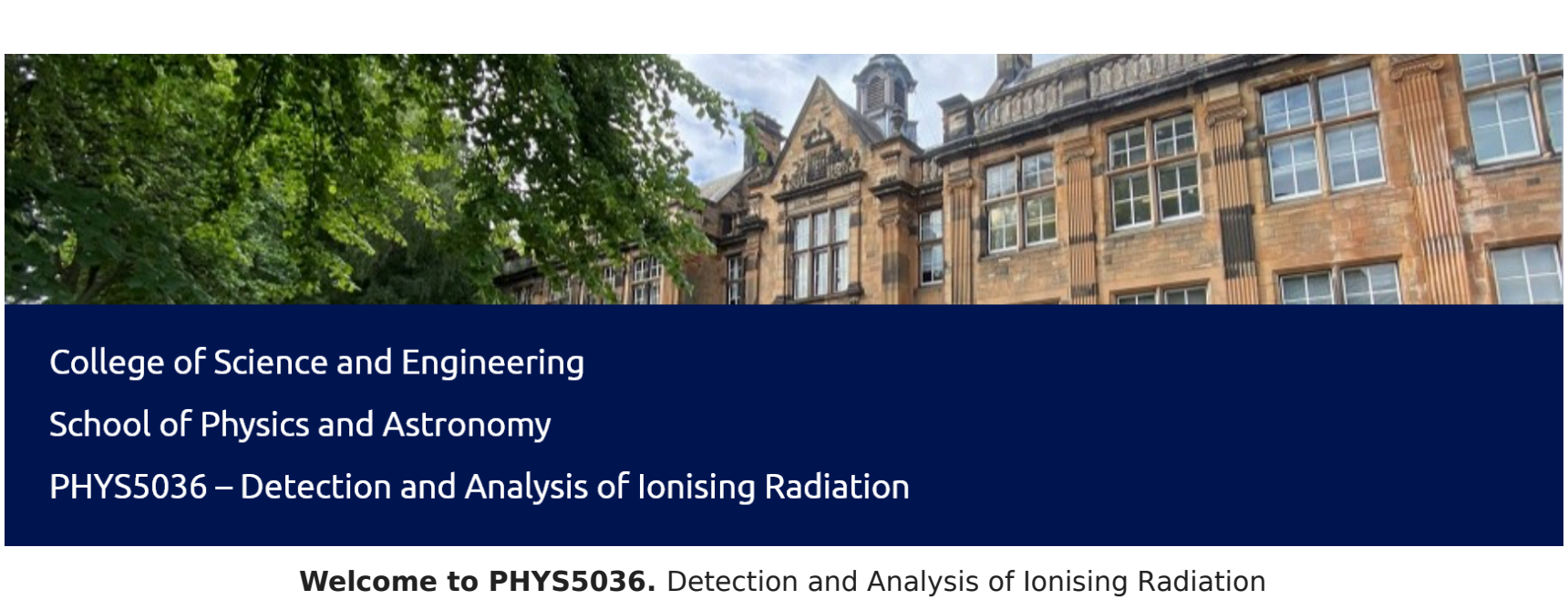

College of Science and Engineering
School of Physics and Astronomy
PHYS5036 – Detection and Analysis of Ionising Radiation

**Welcome to PHYS5036.** Detection and Analysis of Ionising Radiation

In [ ]:
```python
# Importing the required modules
import pandas as pd                  # Common python dataframe package used in industry and academia
import matplotlib.pyplot as plt      # Common python package for plotting
from pathlib import Path
```

# Analysis of Dose Distributions in Radiotherapy Experiment

This is a computational-based experiment using a Monte-Carlo physics simulation software called Geant4.

In nuclear and particle physics research, it is very common to perform physics simulations to refine methods and demonstrate feasibility before costly experimental tests.

One sub-field of cutting-edge research is in beam radiotherapy, where gamma, electron, proton and ion beams can be used to target cancerous regions.

The aim of this virtual experiment is to investigate how beams of different particles types and different energies deposit energy in a simplified setup using a water phantom.

**Objectives:**

1. To gain an understanding of the electromagnetic interactions of high-energy gamma-rays, electrons, protons and ions and relate the range of these particle types in matter to their underlying interactions.
2. To understand the relationship between deposited energy, absorbed dose and equivalent dose.
3. To become familiar with secondary particle interactions and hadronic processes and directly relate them to radiotherapy applications.
4. To gain an appreciation of the differences in dose distributions in targeting cancer with different beam types.

## Software Installation Instructions

You must complete this step before proceeding with the lab. Please ask a demonstrator as soon as possible if you run into any issues.

Before we can start investigating radiotherapy with different beam types, the software must first be downloaded, set up correctly and compiled on the School's UNIX teaching server.

1. Follow the steps in the "First Computing Set Up Steps" document, found on the Lab Moodle page, under the Lab Material ➔ Computing section. It provides some useful introductory reading for using the School's computing.

   Once you are happy with that document, log into JupyterHub in the normal way (remember if you are working remotely, you will need to first connect via VPN).

2. Then follow the environment set up instructions in Section 2 of the "Experiment Computing Set Up" document from on the Lab Moodle page, under the Lab Material ➔ Computing section.

   If you have any questions during this setup stage speak to the lab staff, who are happy to help you.

   When we execute the command `source /local/cern/software/Sim/SoftwareSetup.sh` during the Experiment Computing Set Up, we are telling your UNIX environment how to find the relevant simulation code. This is a Geant4 particle transport Monte Carlo simulation called DoseProfileMC which has been designed specifically for this laboratory.

Advanced: For students interested in the Geant4 software details, the C++ class definition files can be found on Moodle in the Computing section, in a folder called "DoseProfileMC Source Code" Have a look at the folders and files inside. The class implementation files are in `DoseProfileMC/src/` subdirectory. Look through these classes if you are curious how the C++ code for a Geant4 simulation is run and ask if you have any questions. The physical geometry of the setup is defined in `src/DetectorConstruction.cc`.

User control of the simulation can be implemented by editing text files like those found in the `macros/` subdirectory: `DetectorSetup.mac`, `vis.mac` and `batch.mac`. Have a look at the commands in these files if you are interested in how Geant4 is run.

## Background Physics

Before starting this lab, it is highly recommended to have a read of the background material on the moodle page. You will be working with gamma rays and heavy charged particles (protons, C-12 ions), so refer to the relevant sections on how each of these particle types interact with matter to help you interpret your results. Also provided in this document is some information on Monte-Carlo generators.

It is further encouraged you do some background research of your own and converse with the demonstrators for further understanding of aspects related to the lab outwith this material.

Questions:

- What are the three main types of interactions that gamma rays can have with matter, and how do these interactions depend on the energy of the gamma rays?
- How do light and heavy charged particles interact with matter, and how does this depend on their energy?
- What are the main benefits of using Monte-Carlo simulations over an experimental approach?
- How does Geant4 reliably simulate particle interactions with matter, how is this deterministically performed, and what role do random numbers play in the simulation? Hint: This may require some extra reading on the Monte-Carlo method and the implementation details of Geant4.

## Introduction: Radiotherapy for Cancer Treatment

More and more treatment plans for cancer patients involve the use of radiotherapy, in which a beam of high-energy subatomic particles is used to cause catastrophic cell damage in cancerous tumours. The damage is done at the DNA level through microscopic interactions in which the beam particles transfer energy to the tumour tissue causing bonds to be broken and eventually cellular death. In order that the dose to which the tumour is exposed is much larger than the dose received by healthy tissue, treatment plans normally involve the use of shaped beams targeting the tumour area from several different orientations as shown in Figure 1.

While the effectiveness of such treatment has been demonstrated in the clinical setting, there is a large international research effort into using different types of beams. Much of the motivation for this research is due to the need to further minimise the damage caused to healthy tissue during these types of treatments.

There has therefore been a paradigm shift away from the use of high-energy gamma-ray beams to the use of hadron (proton or heavy ion) beams. Indeed, the NHS in England is in the process of developing two new hadron therapy treatment facilities. The shift from gamma to hadron beams is intimately connected to the nature in which these different particles interact and deposit dose in the patient. The motivation for the current virtual experiment is exploring the relevant particle interactions in this radiotherapy context.

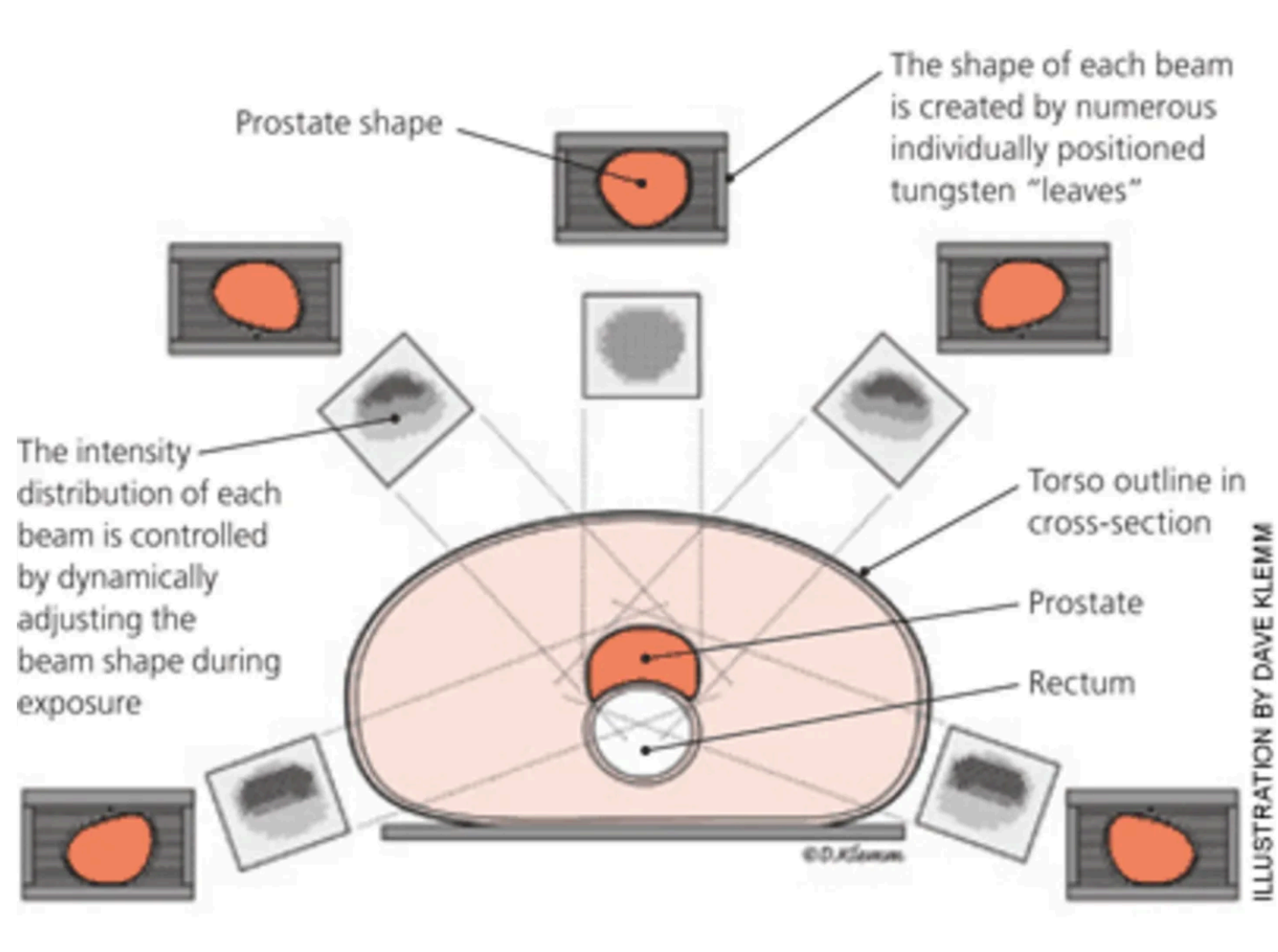

Prostate shape
The shape of each beam is created by numerous individually positioned tungsten "leaves"
The intensity distribution of each beam is controlled by dynamically adjusting the beam shape during exposure
Torso outline in cross-section
Prostate
Rectum
©D.Klemm
ILLUSTRATION BY DAVE KLEMM

**Figure 1:** An example showing the use of several beam shapes, orientations and intensities. Taken from http://www.aafp.org/afp/2008/1201/p1254.html .

---

## Part One: The Electromagnetic Interactions of Gamma rays

---

The energy-loss process for high-energy gamma-rays or charged particles is dominated by electromagnetic interactions between the incident particle and atomic electrons in matter. The nature of these interactions is dependent on particle type (charge and mass) and energy. In radiotherapy, it is these electromagnetic interactions that play by far the largest part in governing the nature in which energy is transferred from the beam particle to the patient.

We will investigate the energy dependence of the electromagnetic interactions for the four different beam types of interest (gamma-rays, electrons, protons and ions) by simulating a beam incident on a water phantom.

To begin with we will generate a beam of 1 MeV gamma-rays. This is a good starting point as radiotherapy is currently dominated globally by treatment plans and technologies involving high energy gamma-rays.

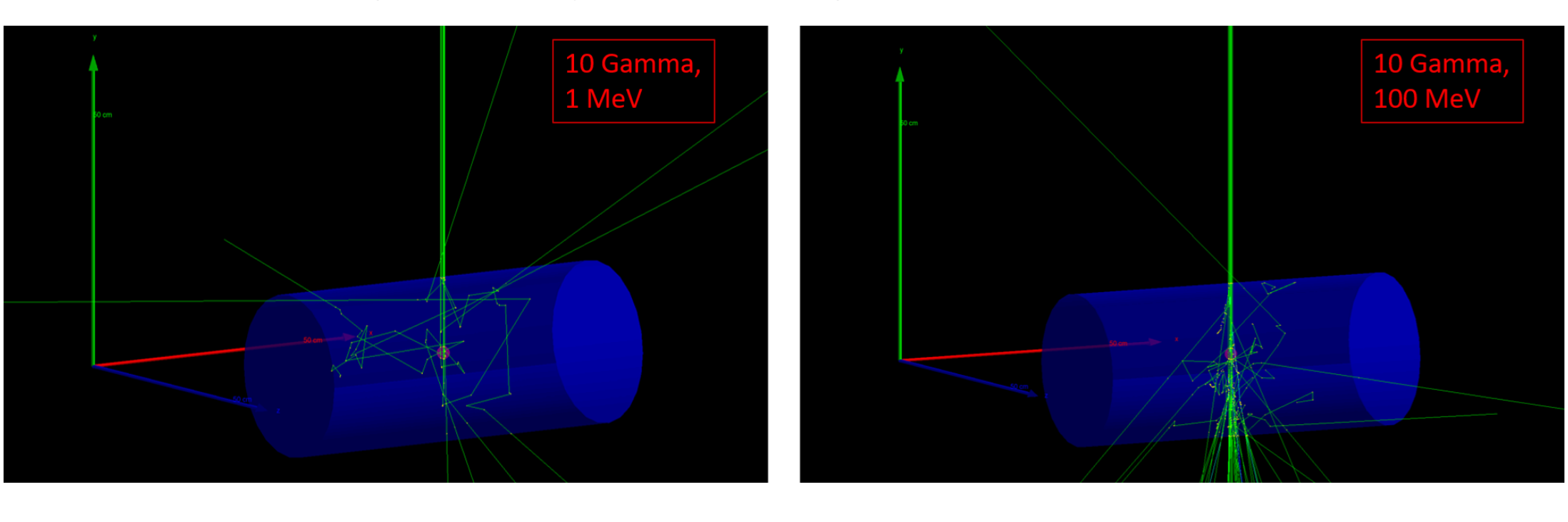
### 1.1 Running your First Simulation

The python script `runDoseProfile.py` will be used for executing all simulations in this lab. This should be ran in a terminal window. The script takes 5 arguments in the following order: `runDoseProfile.py <num_particles> <Tumor_flag> <Hadronic_flag>.

We will start by running a simple simulation with a gamma beam.

```
runDoseProfile.py gamma 1 10000 0 0
```
The command uses a 1 MeV gamma beam, 10,000 incident particles, no tumour, and electromagnetic physics only. The output CSV should be available in the directory configured by `OUTPUT_DIR`.

The visualisation shown in Figure 2 is provided as a reference; it is not run through JupyterHub. The gamma beam travels along the -y direction. The transverse direction is the x-z plane, and the longitudinal direction is the y-axis.



**Questions**

1. Based on Figure 2, what observations can be made about the interaction between the beam and the phantom?
2. Using your background knowledge of gamma-ray interactions with matter, explain why the tracks appear differently at different energies.

### 1.2 Analysing the Simulated Data

It is of course important to be more rigorous in these investigations than simply visualising the interactions of a handful of gamma-rays. We do this by investigating in detail how energy is deposited in the phantom by analysing the simulated data using python.

The output from the simulation is written to the directory specified by `OUTPUT_DIR`. For the example command above, the filename is `DoseMC_gamma_1MeV_10000n_Out.csv`. Opening the csv file may be slow, since it is a lot of data. The phantom is split up into cubic volume elements (voxels) in order to discretise the data, Figure 3 below shows a visualisation of how the voxel grid is setup in the phantom.

The csv file records the energy deposited in each voxel, as well as the type of particle(s) which deposited the energy. Ignore the columns referring to "T" for the moment, these will become apparent later in the script. Figure 3 below shows a visualisation of where the voxel grid is defined in the phantom.

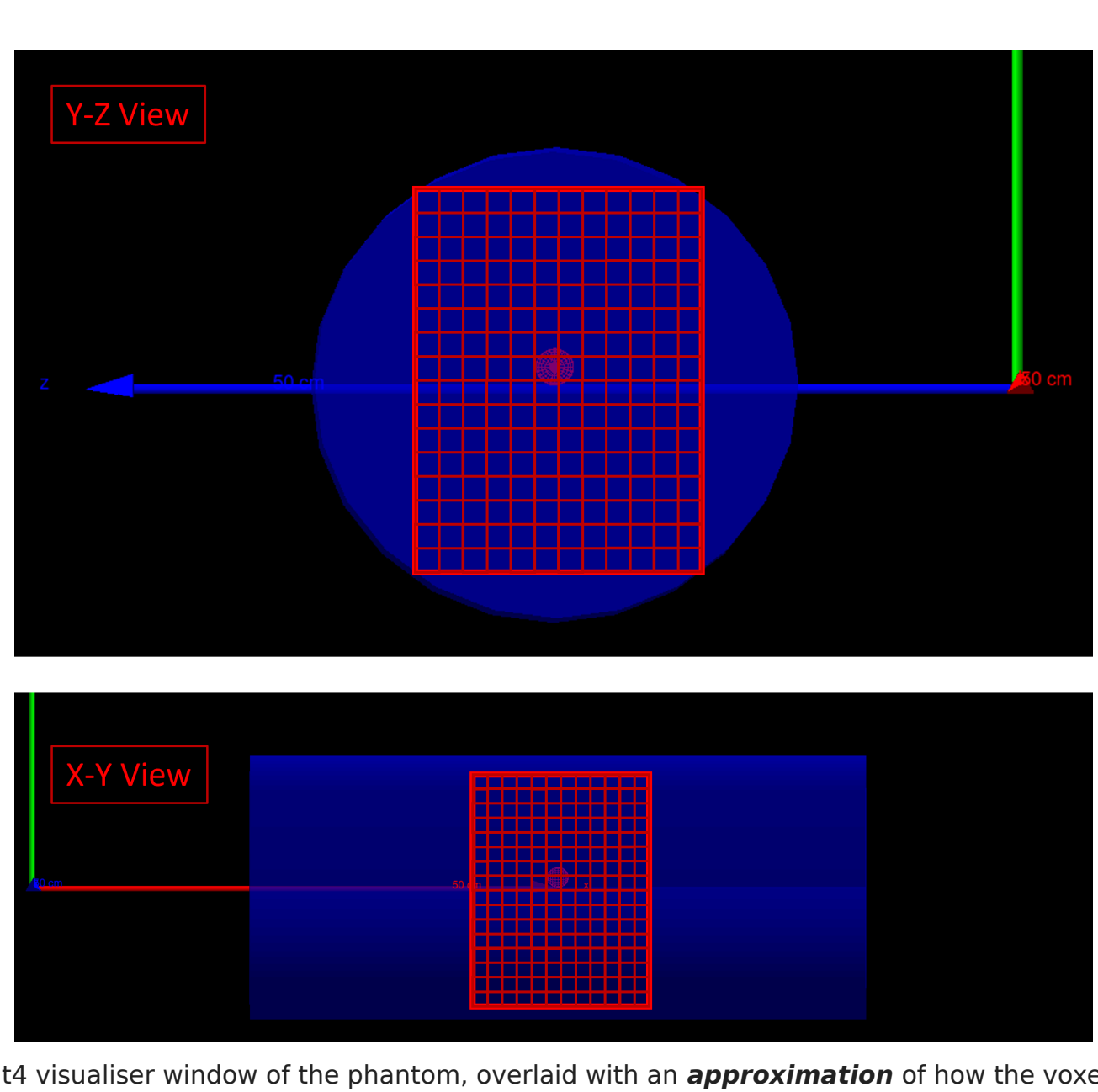


**Figure 3:** Geant4 visualiser window of the phantom, overlaid with an ***approximation*** of how the voxel grid is defined.

We have provided some code within this notebook to help you get started with the analysis. You can modify and expand this code however you wish. This includes a read-in function alongside some analysis/plotting functions of the data in the below sub-sections.

#### 1.2.1 Reading in the Data

In [ ]:
```python
def read_process_csv(fileName):

    # First, we read the header of the csv file to get metadata
    header = pd.read_csv(fileName,nrows=1)


    xbins  = header["xbins"][0]
    xmin   = header["xmin"][0]
    xmax   = header["xmax"][0]

    ybins  = header["ybins"][0]
    ymin   = header["ymin"][0]
    ymax   = header["ymax"][0]

    zbins  = header["zbins"][0]
    zmin   = header["zmin"][0]
    zmax   = header["zmax"][0]

    bins = [xbins, ybins, zbins]
    rangemin = [xmin, ymin, zmin]
    rangemax = [xmax, ymax, zmax]

    intnev   = header["nevent"][0]
    particle = header["particle"][0]
    energy   = header["energy"][0]

    # Second, we read the data of the csv file and organise into a Pandas Dataframe
    dataframe = pd.read_csv(fileName,skiprows=2)
    dataframe["sum"]     = dataframe["lepton"]+dataframe["meson"]+dataframe["baryon"]+dataframe["ion"]
    dataframe["sumT"]    = dataframe["leptonT"]+dataframe["mesonT"]+dataframe["baryonT"]+dataframe["ionT"]
    dataframe["leptonA"] = dataframe["lepton"]+dataframe["leptonT"]
    dataframe["mesonA"]  = dataframe["meson"]+dataframe["mesonT"]
    dataframe["baryonA"] = dataframe["baryon"]+dataframe["baryonT"]
    dataframe["ionA"]    = dataframe["ion"]+dataframe["ionT"]
    dataframe["Total"]   = dataframe["sum"]+dataframe["sumT"]
    dataframe["xpos"]    = xmin+(0.5+dataframe["x"])*(xmax-xmin)/xbins
    dataframe["ypos"]    = ymin+(0.5+dataframe["y"])*(ymax-ymin)/ybins
    dataframe["zpos"]    = zmin+(0.5+dataframe["z"])*(zmax-zmin)/zbins

    return dataframe, intnev, particle, energy, bins, rangemin, rangemax
```

#### 1.2.2 Dose Profile by Particle Type

The particle type in these plots refers to the particle that deposited the energy, not the incident beam. Depositors are grouped as leptons, mesons, baryons or ions. The incident ion beam used in this lab is a carbon-12 (C-12) ion; its primary-particle name appears as `C12` in the CSV metadata.

Simulation CSV files are written to the directory configured by `OUTPUT_DIR`, with names of the form `DoseMC_<particle>_<energy>MeV_<events>n_[Tumour_][Hadronic_]Out.csv`; the optional suffixes are included when those flags are enabled. The longitudinal scoring grid covers the full 260 mm phantom diameter, from y = -130 to +130 mm, using 1 mm bins. When a plotting function is called with `savefig=True` and a non-empty `tag`, it saves its PDF in the `plots` subfolder of the Jupyter kernel's current working directory, creating that folder if needed. The filenames are `DoseByParticle_<tag>.pdf`, `LongitudinalProfile_<tag>.pdf`, `TransverseProfile_<tag>.pdf` and `TransverseProfile2D_<tag>.pdf`. The plots are also displayed by `plt.show()`.

A tag is a short label added to each plot filename so plots from different simulations do not overwrite one another. `simulation_tag(csv_path)` automatically derives one from the CSV filename (for example, `DoseMC_ion_1000MeV_10000n_Out.csv` becomes `ion_1000MeV_10000n_Out`). You can also provide your own label, such as `tag='C12_1000MeV'`. Pass the same tag to each plotting function for one simulation. If you omit the tag or leave it empty, the generic PDF name is saved in the current working directory instead of `plots/`.

The `plot_components_as_hist` function creates a histogram (frequency distribution) of deposited energy for different particle types.

The `calculate_energy_by_particle` function calculates the fraction of the total energy deposited by each particle type and prints out the energies deposited by the particles.

In [ ]:
```python
def simulation_tag(filename):
    """Return a filename-safe tag derived from a simulation CSV path."""
    return Path(filename).stem.replace('DoseMC_', '')


def plot_output_path(plot_name, tag):
    filename = f'{plot_name}_{tag}.pdf' if tag else f'{plot_name}.pdf'
    if tag:
        plots_dir = Path('plots')
        plots_dir.mkdir(parents=True, exist_ok=True)
        return plots_dir / filename
    return Path(filename)


def plot_components_as_hist(dataframe, nbins=100, savefig=False, tag=''):
    if dataframe.empty or not dataframe['Total'].gt(0).any():
        print('No positive energy-deposition voxels; skipping particle histogram.')
        return

    # Plot deposited energy by particle type.
    erange = 0, dataframe["Total"].max()

    plt.figure(figsize=(15, 10))
    dataframe['leptonA'].hist(bins=nbins, range=erange, color='black', alpha=0.5, label='Leptons')
    dataframe['mesonA'].hist(bins=nbins, range=erange, color='red', alpha=0.5, label='Mesons')
    dataframe['baryonA'].hist(bins=nbins, range=erange, color='blue', alpha=0.5, label='Baryons')
    dataframe['ionA'].hist(bins=nbins, range=erange, color='magenta', alpha=0.5, label='Ions')
    plt.xlabel('Energy Deposited (MeV)')
    plt.legend()
    plt.yscale('log')

    if savefig:
        plt.savefig(plot_output_path('DoseByParticle', tag), bbox_inches='tight')
    plt.show()


def calculate_energy_by_particle(dataframe, intnev, particle, energy):
    # Calculate energy deposited by each particle type.
    print('For', intnev, 'incident', particle, 'with an energy of', energy, 'MeV')
    calc_total = dataframe['Total'].sum()
    print('The total energy deposited by ALL leptons, mesons, baryons and ions ', calc_total, 'MeV')

    if calc_total == 0:
        print('Particle fractions are undefined because no energy was deposited.')
        return

    for column, label in [
        ('leptonA', 'leptons'),
        ('mesonA', 'mesons'),
        ('baryonA', 'baryons'),
        ('ionA', 'ions'),
    ]:
        deposited = dataframe[column].sum()
        print(f'Fraction of energy deposited by {label} = ', deposited / calc_total * 100, ' %')
```

### 1.2.3 Longitudinal Dose Profile

This `plot_longitudinal_profile' function plots a 1-D distribution of energy deposited in a central slice through the phantom in the longitudinal (parallel to the beam) direction.

In [ ]:
```python
def plot_longitudinal_profile(dataframe, bins, rangemin, rangemax, intnev, savefig=False, tag=''):
    rangey = (rangemin[1], rangemax[1])
    ybins = int(bins[1])

    cutframeL = dataframe[(dataframe['zpos'].abs() < 10) & (dataframe['xpos'].abs() < 10)]

    plt.figure(figsize=(15, 10))
    plt.hist(cutframeL['ypos'], weights=cutframeL['sum'] / intnev, bins=ybins, range=rangey, alpha=0.5, label='Normal')
    plt.hist(cutframeL['ypos'], weights=cutframeL['sumT'] / intnev, bins=ybins, range=rangey, alpha=0.5, label='Tumour')
    plt.xlabel('Longitudinal Position (mm)')
    plt.ylabel('Energy deposited per incident particle (MeV)')
    plt.legend()

    if savefig:
        plt.savefig(plot_output_path('LongitudinalProfile', tag), bbox_inches='tight')
    plt.show()

    total_sum = cutframeL['sum'].sum() + cutframeL['sumT'].sum()
    mean_voxel_energy = total_sum / (intnev * ybins)
    print('Mean energy deposited per voxel in longitudinal direction ', mean_voxel_energy, 'MeV per voxel')

    normal_sum = cutframeL['sum'].sum()
    tumour_sum = cutframeL['sumT'].sum()
    if normal_sum:
        print('TNR is ', tumour_sum / normal_sum)
    else:
        print('TNR is undefined because the normal-tissue dose is zero')
```

#### 1.2.4 Transverse Dose Profile

Below are the functions for plotting the 1-D and 2-D distributions of energy deposited in a central slice through the phantom in the transverse (perpendicular to the beam) direction.

In [ ]:
```python
def plot_transverse_profile(dataframe, bins, rangemin, rangemax,
                            intnev, savefig=False, tag=''):
    rangex = rangemin[0], rangemax[0]
    xbins = int(bins[0])
    zbins = int(bins[2])

    cutframeT = dataframe[dataframe['ypos'].abs() < 10]

    plt.figure(figsize=(15, 10))
    plt.hist(
        cutframeT['xpos'], weights=cutframeT['sum'] / intnev,
        bins=xbins, range=rangex, alpha=0.5, label='Normal'
    )
    plt.hist(
        cutframeT['xpos'], weights=cutframeT['sumT'] / intnev,
        bins=xbins, range=rangex, alpha=0.5, label='Tumour'
    )
    plt.xlabel('Transverse Position (mm)')
    plt.ylabel('Energy deposited per incident particle (MeV)')
    plt.legend()

    if savefig:
        plt.savefig(plot_output_path('TransverseProfile', tag), bbox_inches='tight')
    plt.show()

    total_sum = cutframeT['sum'].sum() + cutframeT['sumT'].sum()
    mean_voxel_energy = total_sum / (intnev * zbins)
    print('Mean energy deposited per voxel in transverse direction ',
          mean_voxel_energy, 'MeV per voxel')


def plot_2d_transpose_profile(dataframe, bins, rangemin, rangemax,
                              savefig=False, tag=''):
    xbins = int(bins[0])
    zbins = int(bins[2])
    rangexz = ([rangemin[0], rangemax[0]], [rangemin[2], rangemax[2]])

    plt.figure(figsize=(20, 15))
    plt.hist2d(
        x=dataframe['xpos'], y=dataframe['zpos'], weights=dataframe['Total'],
        bins=(xbins, zbins), range=rangexz, cmap=plt.cm.jet
    )
    cbar = plt.colorbar()
    plt.xlabel('X Position (mm)')
    plt.ylabel('Z Position (mm)')
    cbar.ax.set_ylabel('Energy deposited (MeV)')

    if savefig:
        plt.savefig(plot_output_path('TransverseProfile2D', tag), bbox_inches='tight')
    plt.show()


# Backwards-compatible alias for the original notebook name.
plot_transpose_profile = plot_transverse_profile
```

### 1.3 Analysis of the 1 MeV Gamma-ray Beam

Use the functions provided in Section 1.2 to analyse the 1 MeV gamma-ray simulation.

**Questions**

1. What can you say about the magnitude of energy deposited in the phantom?
2. How is the deposited energy distributed longitudinally and transversely?
3. How do these observations relate to the underlying gamma-ray interactions?

In [ ]:
```python
# First set your file name here.
gamma_1MeV_fname = './data/DoseMC_gamma_1MeV_10000n_Out.csv'

# Then read in and process the data.
dataframe, intnev, particle, energy, bins, rangemin, rangemax = read_process_csv(gamma_1MeV_fname)

# Remove zero-deposition voxels for the profile plots and particle summary.
filterframe = dataframe[dataframe['Total'] > 0]

# Use the CSV name for a unique label, or replace it with your own (e.g. 'C12_1000MeV').
tag = simulation_tag(gamma_1MeV_fname)

if filterframe.empty:
    print('No voxels contain deposited energy; check the simulation output before plotting.')
else:
    calculate_energy_by_particle(filterframe, intnev, particle, energy)
    plot_components_as_hist(filterframe, nbins=100, savefig=True, tag=tag)
    plot_longitudinal_profile(
        filterframe, bins, rangemin, rangemax, intnev, savefig=True, tag=tag
    )
    plot_transverse_profile(
        filterframe, bins, rangemin, rangemax, intnev, savefig=True, tag=tag
    )
    plot_2d_transpose_profile(
        filterframe, bins, rangemin, rangemax, savefig=True, tag=tag
    )
```

### 1.4 Investigating the Energy Dependence of Gamma-ray Interactions

Co-60 sources produce gamma rays with energies of 1.173 and 1.333 MeV. Accelerator-based radiotherapy produces higher-energy gamma-ray beams through bremsstrahlung.

Run the following simulations:

```
runDoseProfile.py gamma 5 10000 0 0
runDoseProfile.py gamma 10 10000 0 0
runDoseProfile.py gamma 50 10000 0 0
```
Analyse each output using the same functions as for the 1 MeV beam. The expected files are `DoseMC_gamma_5MeV_10000n_Out.csv`, `DoseMC_gamma_10MeV_10000n_Out.csv` and `DoseMC_gamma_50MeV_10000n_Out.csv`.

**Questions**

1. What conclusions can be drawn about Co-60 sources and accelerator-based gamma-ray technology?
2. How does the nature of the underlying physical interactions change as the beam energy increases?
3. What other factors should be considered when assessing the advantages and limitations of accelerator-based technology?

In [ ]:
```python
# your code here
# Hint: repeat the Section 1.3 analysis for the 5 MeV gamma-ray data.
```

In [ ]:
```python
# your code here
# Hint: repeat the Section 1.3 analysis for the 10 MeV gamma-ray data.
```

In [ ]:
```python
# your code here
# Hint: repeat the Section 1.3 analysis for the 50 MeV gamma-ray data.
```

---

## Part Two: Deposited Energy, Absorbed Dose and Equivalent Dose

---

Up to now, when discussing interactions between the beam and the phantom we have been interchanging the terms deposited energy and dose. In order to understand, in a quantitative sense, the efficacy of a particular radiotherapy technology it is important to define formally what is meant by these terms and the relationship between them.

The stochastic effect in terms of cell damage as a result of exposure to ionising radiation is characterised by the equivalent dose $H$, measured in Sieverts (Sv). The equivalent dose absorbed in tissue is defined such that:

$$H = \sum_R W_R D_R,$$

where WR is a weighting factor for radiation type R, which is a measure of the relative biological effectiveness of the radiation type. Nominal values of WR for different particle types as set by the International Committee for Radiation Protection (ICRP) are:

| **Radiation type** | $W_R$ |
|---|---|
| Lepton | 1.0 |
| Meson | 2.0 |
| Baryon | 2.0 |
| Ion | 20.0 |

The absorbed dose D is defined as the mean energy deposited by ionising radiation in matter (Ed) per unit mass, the unit of which is the Gray (J/kg):

$$D = \frac{E_d}{m}$$

The simulation records the energy deposited in a given volume element (or voxel) of the phantom. The header variable of the csv files gives the voxel dimensions (where min and max variables are in mm).

### 2.1 Longitudinal Profile Slice Volume

For the longitudinal dose profile, voxels are combined in such a way to create individual slices of volume lying along the beam direction and the mean energy per slice is output. Calculate the volume of each slice used in the longitudinal dose profile calculation which gives the result called mean energy deposited per voxel.

Using these numbers calculate the total absorbed dose (D) and equivalent dose (H) per incident beam particle for the four gamma-ray energies you just simulated in part one.

In [ ]:
```python
# Calculate the volume represented by one longitudinal-profile slice.
# Show your calculation and report the volume with units.
```

### 2.2 Calculating Absorbed and Equivalent Dose

After analysing the 1, 5, 10 and 50 MeV gamma-ray files in Part One, write a Python function that calculates the absorbed dose ($D$) and equivalent dose ($H$) per incident beam particle from the deposited energy and mass. Use the mass of the full phantom volume represented by the data, convert energy to joules and mass to kilograms, and apply the radiation weighting factors given above. Use your function to calculate $D$ and $H$ for each gamma-ray energy.

In [ ]:
```python
# your code here
# Define your dose-calculation function, then use it to calculate D and H per incident particle
# for the 1, 5, 10 and 50 MeV gamma-ray simulations.
```

### 2.3 Number of 10 MeV Gamma Rays for 50 Gy

Using the 10 MeV result from Section 2.2, estimate how many gamma rays would need to be incident on the phantom to deliver an absorbed dose of 50 Gy to the full phantom volume. Assume dose scales linearly with the number of incident particles.

In [ ]:
```python
# Your calculation here.
```

### 2.4 Dose in the Central Beam Region

Looking at the 2D distribution of x-z slice energy deposits, estimate a rectangle which contains the core of the energy deposited by the beam. Filter the dataframe to select only these voxels and recalculate $D$ and $H$, using the mass represented by the beam-core voxels rather than the full phantom. For example:

```python
newframe = filterframe[
    (filterframe['zpos'].abs() < 10) &
    (filterframe['xpos'].abs() < 10)
]
```
You may find it easier to create a new function to filter the beam-core voxels.

**Questions**

1. How do the full-phantom and beam-core absorbed and equivalent doses compare?
2. Why does the dose change when only the beam-core voxels are included?

In [ ]:
```python
# your code here
# Select beam-core voxels, recalculate dose using the core mass, and compare with the full phantom.
```

---

## Part Three: Electromagnetic Interactions of Heavy Charged Particles

---

In this part, "ion" means the carbon-12 (C-12) ion used as the heavy-ion beam. Electromagnetic energy loss for heavy charged particles is dominated by the ionisation process as described by the Bethe-Bloch equation, in which the ionisation density is related to the charge, mass and velocity of the incident particle. As the particle slows, the cross-section for interactions with atomic electrons increases leading to higher and higher energy loss until the particle comes to rest. It is this behaviour, characterised by the presence of a Bragg peak in the dose distribution for heavy charged particles, that has led to intense interest in proton and ion radiotherapy in recent years.

### 3.1 Simulation of Proton and Carbon-Ion Beams

Here, "ion" means the C-12 (carbon-12) ion. The energy argument is passed to Geant4 as the ion's kinetic energy in MeV. In order to demonstrate the significant difference between gamma-ray and hadron radiotherapy, we are going to run the simulation with beams of protons and carbon ions. As before, let's simulate and extract the dose distributions for proton beams with kinetic energies of 100, 500 and 1000 MeV. Now let's do the same for C-12 ion beams of 500, 1000 and 2000 MeV.

Use the runDoseProfile.py script to run these simulations. Make sure to specify the particle type (proton or ion) and the desired kinetic energy for each run, for example: `runDoseProfile.py proton 100 10000 0 0`.

In [ ]:
```python
# 100 MeV Proton
```

In [ ]:
```python
# 500 MeV Proton
```

In [ ]:
```python
# 1000 MeV Proton
```

In [ ]:
```python
# 500 MeV Ion
```

In [ ]:
```python
# 1000 MeV Ion
```

In [ ]:
```python
# 2000 MeV Ion
```

### 3.2 Analysis of Proton and Carbon-Ion Dose Distributions

The ion simulations in this section use C-12 (carbon-12) ions. You should now have six dose distribution files corresponding to the proton and C-12 ion simulations you ran. Proceed to plot the dose distributions for each below. For tidiness, keep one file to one cell.

Once you have plotted all the dose distributions, have a think about the following questions.

**Questions**

1. By looking at the resultant plots, what can you conclude about the nature of the energy deposit for hadron beams?
2. What can you say about the range of the charged particles in the water phantom, and how might this be exploited in cancer treatment?
3. How does the longitudinal and transverse dose profile differ from the gamma-ray data?
4. Comment on the typical length scale (i.e. FWHM) associated with the Bragg peaks for protons and ions of the same energy. What implications does this latter observation have in terms of treatment planning?

In [ ]:
```python
# your code here
```

---

## Part Four: Realistic Radiotherapy Simulations with Hadronic Interactions

---

## Part Four: Realistic Radiotherapy Simulations with Hadronic Interactions

In Part Four, the ion beam still means C-12 (carbon-12). The investigations up to now have focussed on electromagnetic interactions between the beam particles and the phantom. In order to fully understand the dose profiles in radiotherapy applications, we must also include in the simulation hadronic interactions between the beam and phantom. Take a short time to look into what is meant by hadronic interactions and how they lead to a higher dose being deposited in the phantom.

Before including these interactions, we are first going to make the radiotherapy simulation more realistic by placing a small sphere (1 cm diameter) at the centre of the phantom. (we add this by setting the tumor flag to 1 when running the simulation, e.g., `runDoseProfile.py gamma 1 10000 1 0`). By recording the energy deposited in this sphere separately from the rest of the phantom, we can start to quantify the effectiveness of different therapy treatments. Note this corresponds to the columns with a "T" identifier in the csv file. To quantify the effectiveness, we consider the sphere to be the tumour that is being targeted and the rest of the phantom to represent healthy tissue. This then allows us to simulate and measure the Tumour-to-Normal (TNR) ratio, which as the name suggests is simply:

$$\mathrm{TNR} = \frac{D_{\mathrm{tumour}}}{D_{\mathrm{healthy}}}$$

The ultimate goal in radiotherapy treatment planning is keeping TNR as high as possible in order to minimise the damage sustained in the patient's normal or healthy tissue.

### 4.1 Tumour-to-Normal Dose Ratio

For this section, the ion beam means C-12 (carbon-12). Now re-run your previous simulations to include the spherical tumour volume, setting the [TumourFlag]=1 when executing the simulation script. Do this for all selected beam types and energies.

**Questions**

1. Explain the key differences between each beam type in terms of their TNR.
2. Why are the beam types optimised at different energy ranges?
3. Estimate the optimum beam energy for each beam type based on your TNR results (don't run any additional simulations just yet).

### 4.2 Maximising the Tumour-to-Normal Dose Ratio for each beam type.

The ion beam in this section is C-12 (carbon-12). As the goal of radiotherapy is to find maximum tumour-to-normal dose ratios, you should analyse the TNR results for each beam type and energy combination to determine the optimal settings. This may require some trial and error, but use your estimates from section 4.1 to guide your initial choices.

Hint: Once you have sampled a good range of beam energies, think about analytical ways of finding the maximum TNR for each beam type.

In [ ]:
```python
# your code here
```

### 4.3 Enabling Hadronic Interactions

The ion beam in this section is C-12 (carbon-12). Once you have found optimal beam energies for each beam type, we will now repeat the simulations but this time with hadronic interactions enabled. This is set with the hadronic interaction flag, e.g. `runDoseProfile.py gamma 1`

`10000 1 1`. This changes the Geant4 physics configuration from `standard_opt3` (EM interactions) to `QGSP_BIC_EMY` (EM and hadronic interactions). Repeat your Section 4.2 analysis to find the optimal TNR for each beam type with hadronic interactions enabled.

**Questions**

1. What do we mean by hadronic interactions, and how does this change what the simulation is doing?
2. How does enabling hadronic interactions affect the optimal TNR for each beam type?
3. How does enabling hadronic interactions affect the dose distributions for each beam type?
4. Which beam type is most affected by enabling hadronic interactions? Why?
5. The addition of hadronic interactions means that you should see more secondary particle production. How do thesesecondary particles contribute to the dose magnitudes and distributions?

```
In [ ]: # your code here
```

Original script by D. Hamilton. Revisions by R. Montgomery and W. O'Donnell.